Data Manager Suite - Complete Map

LEONARDO V2

Scope, architecture, complete workflows, object lifecycles and Leonardo integration

A visual companion to the documentation
Use this map alongside the four Data Manager manuals. It adds a navigable system view; the manuals retain the complete commands, lifecycle rules, field inventory, algorithms and concrete format examples.

Data Manager is Leonardo's workspace for persisted research products: portable definitions, managed calculations, reusable collections, owned Seeds and immutable Database revisions. It coordinates existing authorities through explicit, reviewable operations.

Edition basis	Verified reference
Prepared	Original: 8 October 2026, edition 1.0. Targeted R07 revision: 28 October 2026.
Working checkout	main at deb24138042d2de524f38d0b230c1aad7611f83e plus accepted uncommitted work.
Runtime	Python 3.12.12 / PySide6 6.10.2
Relationship to Research	Matching documentation structure and map design. Research map unchanged; its lifecycle manual has the targeted R07 reconciliation.
Authority	Current code and canonical models, supported by completed DMS audits. Historical reports retain their original scope and date.
Reference design	Research Suite Complete Map, following the earlier Core map. Domain facts are verified against current DMS sources.

This edition makes no application, architecture, financial rule, persistence format or real-data change. Page 24 records the historical long-path finding and its R02 code/package resolution; native acceptance remains separate.

Reading guide and navigation

Reference	Companion manual in this folder
S - Scope and active interface	DATA_MANAGER.md
A - Architecture and integration	DATA_MANAGER_ARCHITECTURE.md
L - Workflows and lifecycles	DATA_MANAGER_LIFECYCLES.md
D - Data and interface contracts	DATA_MANAGER_DATA_CONTRACTS.md

03 Scope and eight object roles
04 Active production composition

05 Authorities and task execution
06 Identity and compatibility

07 Catalogs and OHLCV selection
08 Operation checks and stale plans

09 Environments and portable Recipes
10 Recipe Collection lifecycle

11 Direct creation and materialization
12 Batch Construct expansion

13 Managed Artifact versions and updates
14 Artifact Collection lifecycle

15 Seed creation and append
16 Seed-only Database creation

17 Database membership and ownership
18 Append and truthful legacy history

19 Removal and deletion survival
20 Reconciliation and currentness

21 Cancellation and window lifecycle
22 Integration with Leonardo

23 Persistence and reconstruction
24 Evidence and present limitations

Diagram key: blue = interaction or coordination; green = owned domain outcome; amber = explicit decision or proof boundary; red = refusal. Arrows describe the labeled dependency, data or lifecycle direction.

Scope and eight object roles

DMS brings persisted research products into one inspection and operation workspace. A shared catalog does not make DMS the owner of every object shown there.

Role	Meaning / canonical owner
1. OHLCV dataset	Accepted canonical market candles and validation evidence. OHLCV owns acquisition, validation and repair.
2. Study Environment	Reusable ordered Study definitions. Research owns the saved Environment; DMS can derive portable definitions.
3. Portable Recipe	Global market-independent executable definition: tool, semantic parameters and actual inputs. Recipe domain.
4. Recipe Collection	Reusable roots plus complete Recipe dependency membership. Immutable revisions and a current head.
5. Managed Artifact	Market-bound saved calculation with embedded recipe, exact values, physical versions and logical identity. Artifact domain.
6. Artifact Collection	Market-bound snapshot of selected roots, supports and outputs. Explicit revisions; DMS creation store.
7. Database Seed	New V2 Seeds own full-source OHLC and optional Volume. Legacy V1 remains reference-based.
8. Database	Owned values, immutable definition/revisions and a mutable head. Explicit membership edits and append.

Outside this scope
DMS does not acquire candles, redefine formulas, become a Notebook owner, perform trading, invent an Analysis engine, or silently update every descendant of a changed object.

Validity, currentness and ownership are separate. A historical Database can remain valid after its original providers disappear, while future append needs fresh, provable providers.

Active production composition

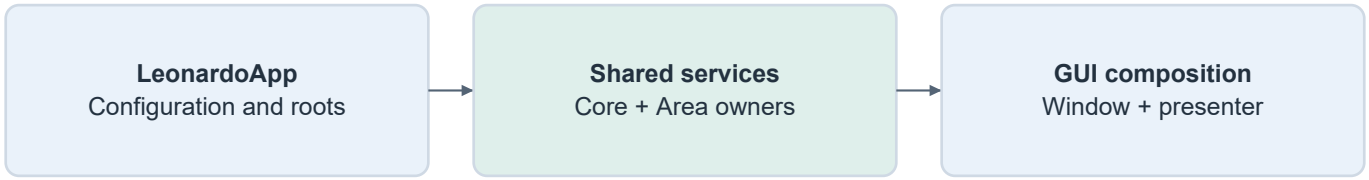


Figure 1. Construction supplies shared dependencies. Windows do not invent stores or roots.

Production component	Responsibility
DataManagerSuiteWindow	Tracked single-instance shell: Catalogs and Update & Reconcile tabs, active dataset, inspector, history and Operation.
DataManagerSuitePresenter	Translates intent, retains bounded dialogs, correlates pending work and applies matching results.
DataManagerApplicationService	Submits long-running work through the shared CoreRunner and returns structured results.
DataManagerService	Coordinates canonical services, checked input sessions and read projections.
Creation / Update workflows	Apply domain policies for explicit creation, composition, publication, reconciliation and updates.
Canonical stores and calculators	Own serialization, identity, durable writes, validation and numerical meaning.

What the user actually sees

Top actions are Refresh, Select Dataset, Preview Dataset and Duplicate Maintenance. Open Notebooks is a Research navigation shortcut. Product commands belong to Catalogs. Database creation is a Catalog action; the retained internal nine-stage creation workspace is not an active third tab.

Preserved shell behavior
A newly constructed Suite starts maximized. Reopening an existing Suite focuses it and preserves geometry. Workspace : Operation = 3 : 1; upper : bottom = 7 : 3; Inspector : Revision History = 1 : 1.

Authorities and task execution

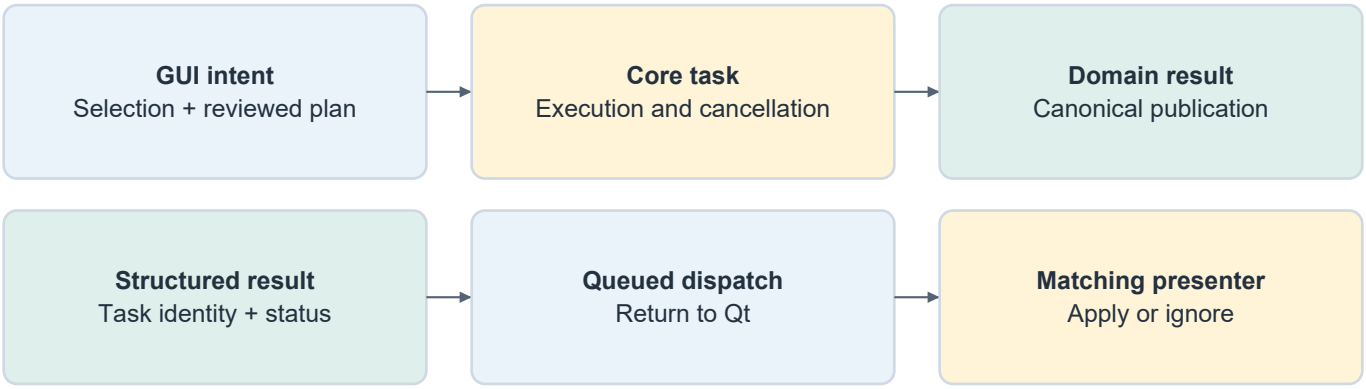


Figure 2. Workers perform domain work; queued GUI delivery checks current ownership before changing presentation.

Truth	Authority
Task identity, progress, terminal state	Core TaskManager; CoreRunner executes work.
Market identity and accepted candles	Canonical MarketId and OHLCV validation/store.
Tool parameters, outputs and formulas	Financial Tools specifications and calculators.
Portable semantic identity	Recipe models, identity functions, graph planner and store.
Artifact values, lineage and heads	ArtifactService and its canonical stores.
Seeds, Collections and Database revisions	DataManagerCreationStore through creation/update policies.
Selection, visible dialog and pending intent	Presenter / Qt state; disposable projections only.

One foreground display, shared Core authority
The Operation panel presents the foreground task. Background reconciliation is separately correlated through the same Core and does not seize foreground progress. No GUI window becomes a second executor.

Identity and compatibility

Identity	What it distinguishes
Canonical MarketId	Exchange + market type + symbol + timeframe; symbol alone is insufficient.
Portable Recipe ID	Global executable definition, independent of market, display name and origin.
Semantic object key	Recursive equivalence of actual inputs and semantic parameters, including older dependency IDs.
Logical Artifact ID	Market + portable definition; stable identity across physical versions.
Physical Artifact ID	Exact calculation recipe, source evidence and saved values.
Object ID / revision ID / head	Stable object, immutable publication, current pointer. Historical inspection never advances a head.

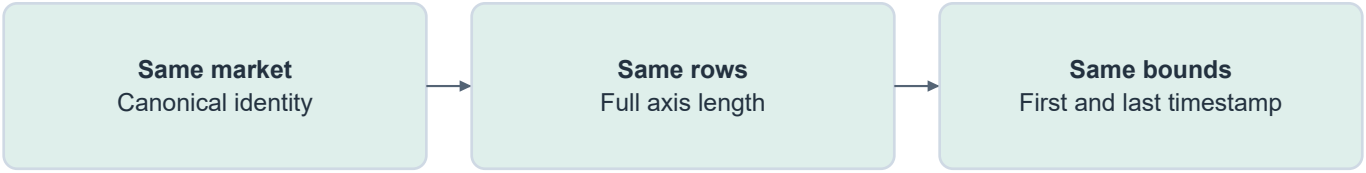


Figure 3. The three materialized admission rules. They do not replace payload, type, graph or integrity validation.

Provenance is not a fourth generic admission rule
CSV/sidecar fingerprints, physical versions and hashes remain exact evidence and reviewed-plan fences. Compatible materialized objects need not pretend they were created from identical historical validation sidecars.

Changing a raw Construct input from Open to Close changes actual calculation identity. Changing a display name or deriving an equivalent Recipe from another Environment does not create new semantic meaning.

PortableRecipeV1 and market-bound ArtifactRecipeV1 are distinct contracts. Reusing the word Recipe does not make their identities or payloads interchangeable.

Catalogs and OHLCV selection

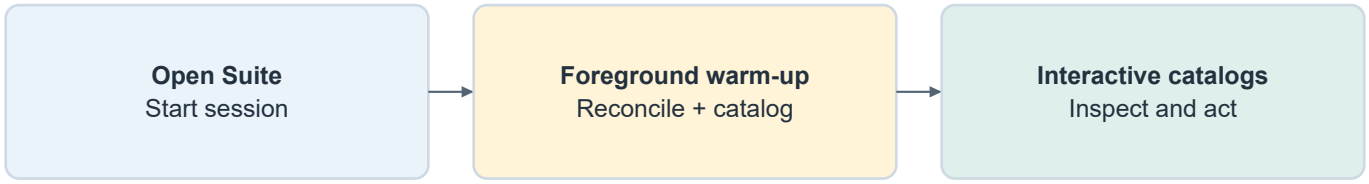


Figure 4. First-open success marks initialization. Failure stays recoverable through Refresh; a reopened live Suite is reused.

Surface	Contract
Dataset selector	Select complete accepted MarketId through search, typed sorting and cascading filters; no symbol-only inference.
Current Dataset scope	Filters market-bound products to the selected dataset. Global definitions remain globally meaningful.
All Datasets scope	Provides access to persisted and orphaned products when the original source is unavailable.
Inspector and revision history	Metadata and historical manifests. Selecting a past revision never publishes or moves a head.
Preview / consuming operation	Loads or checks full values only when that action actually needs them.
Catalog refresh	Metadata-oriented read projections; no hidden full CSV/hash sweep on every click.

Source lifecycle stays with OHLCV

DMS consumes accepted source evidence and responds to source changes. Download, validation, repair and acceptance remain OHLCV operations. A bad source does not authorize DMS to rewrite old snapshots or silently repair candles.

Historical products remain accessible
A missing current source can block calculation or future append without making an owned Seed or Database disappear from inspection. Use All Datasets to reach those saved products.

Operation checks and stale plans

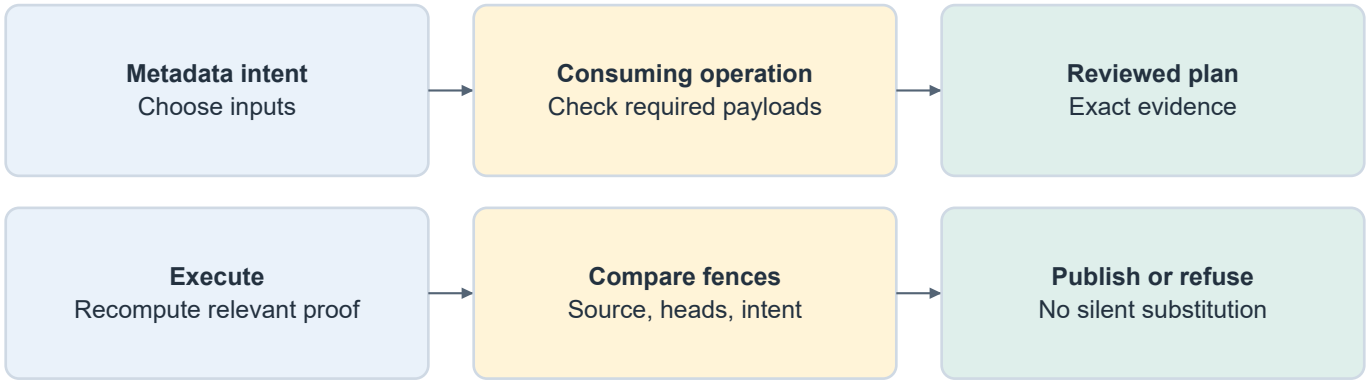


Figure 5. Review and execution share intent but do not assume the world stayed unchanged.

Boundary	Required behavior
DMS check session	Successful checks are scoped to exact session/source generation and source evidence.
Source changes	Advance/invalidate the affected generation. Old success cannot stand for changed inputs.
Failed, cancelled or stale checks	Do not become reusable successful evidence.
Full source check	Parse/hash against accepted metadata under the canonical loader lock when the operation requires it.
Saved Artifact admission	DMS uses its bounded checked context; strict non-DMS consumers retain their policies.
Publication	Recheck exact reviewed dependencies, heads and cancellation at the applicable commit boundary.

No global tamper oracle
The checked session is disposable operation evidence, not a new persistent authority, continuous filesystem watcher or promise to detect every external edit immediately.

Environments and portable Recipes

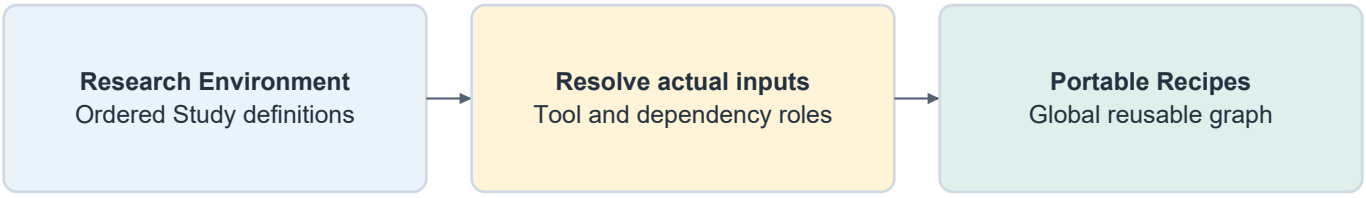


Figure 6. Derivation stores executable definitions and separate provenance; it does not retain the Environment as an execution dependency.

1. Inspect a saved Environment and select the definitions to derive. Research owns its persistence and lifecycle.

2. Resolve canonical Financial Tool parameters, output names and explicit/implicit inputs. Include complete dependency closure.

3. Build portable definitions. Compare recursive semantic meaning so equivalent definitions from different origins can reuse one saved Recipe.

4. Persist immutable Recipe payloads. Record origin, Environment content hash and entry identity separately from calculation identity.

5. Materialize explicitly for a target accepted market when values are wanted. A portable definition alone contains no calculated market values.

Later action	What survives
Edit / delete Environment	Already derived Recipes retain their definitions and provenance.
Record another Recipe origin	Executable identity remains unchanged; origin metadata is separate.
Change actual calculation inputs	Different semantic definition; do not mutate an immutable Recipe in place.
Delete Recipe from global pool	Check true Recipe-domain references first. Valid saved Artifacts retain embedded execution and lineage evidence.

Origin timestamps and first-persisted age are metadata. Missing legacy age remains unknown; it is not guessed from a file modification time.

Recipe Collection lifecycle

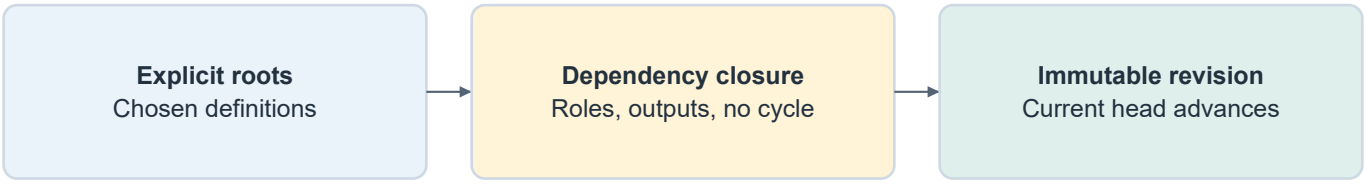


Figure 7. A reusable Collection contains its roots and all required Recipe supports.

Operation	State transition
Create	Validate roots and closure; reuse a semantically equivalent current Collection where defined, otherwise publish a new identity/revision.
Inspect history	Read exact immutable revision; leave the current head unchanged.
Edit membership	Publish a new revision with previous_revision_id. Retain old membership history.
Remove a member	Recompute valid closure. Removing a Collection reference does not delete the Recipe from its pool.
Materialize	Review target market and graph stages, then explicitly calculate/reuse market-bound Artifacts.
Delete Collection	Delete the Collection-owned history/head through its owner; preserve member Recipes.

Historical references are real references
Recipe deletion protection considers retained Collection revisions, not only the newest head. A member removed from today's revision may still be referenced by history.

Definition graph versus result graph

Recipe Collections are portable across target markets. Artifact Collections refer to particular market-bound physical results. Derivation and materialization connect them, but they have separate IDs, stores, validation and lifecycles.

Graph planning refuses missing required definitions, incompatible roles/outputs or cycles. A partial graph is not an executable Collection.

Direct creation and materialization

Entry path	Reviewed intent
Direct Artifact	Choose accepted market, Financial Tool, canonical parameters and permitted raw/saved sources.
Recipe materialization	Choose portable roots or a Recipe Collection, target market and complete planned graph.

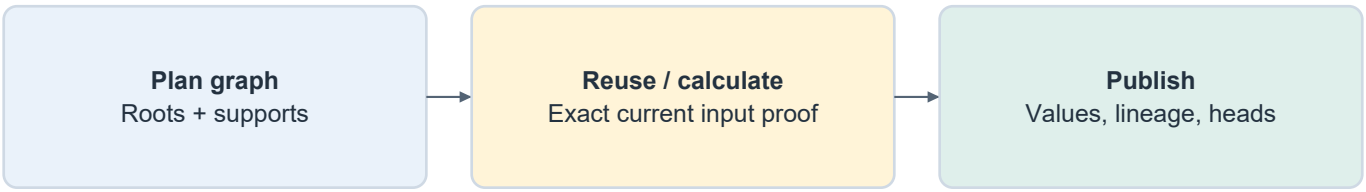


Figure 8. Calculation runs through Financial Tools; ArtifactService owns durable identity and publication.

1. Resolve canonical tool eligibility, input roles and output types. Raw Construct OHLC sources do not include raw Volume.

2. Check required accepted source and saved inputs. Apply the three materialized alignment rules, then enforce payload and lineage validity.

3. Use graph order so supports exist before dependents. Reuse current compatible results when the established plan proves equivalence.

4. Calculate the full source axis. Keep legitimate warm-up NaNs and supported categorical outputs; do not trim to an all-finite intersection.

5. Prepare and publish through ArtifactService with exact source/current-head checks. Return actual created/reused physical IDs and logical version keys.

Sidecar renewal is not automatic recalculation
If the accepted CSV and actual calculation inputs are unchanged, renewing validation metadata alone can reuse a saved Artifact. Historical source evidence remains truthful.

Batch Construct expansion

Batch expands an explicit source selection into individually planned Construct branches. It does not recursively feed newly created branches back into the same selection.

Construct family	Expansion rule
Derivative / angle	One branch per selected signal, role source.
Percent span angle / angle momentum	Automatic expansion uses one branch per signal as source_1; direct multi-input forms preserve numeric role order.
Delta	One fixed fast or slow signal paired with each other selected signal; skip identical pairs.
Braids / braid instability	Explicit fast, mid and slow triples.
Trap area	Explicit fast/slow or fast/mid/slow choices.

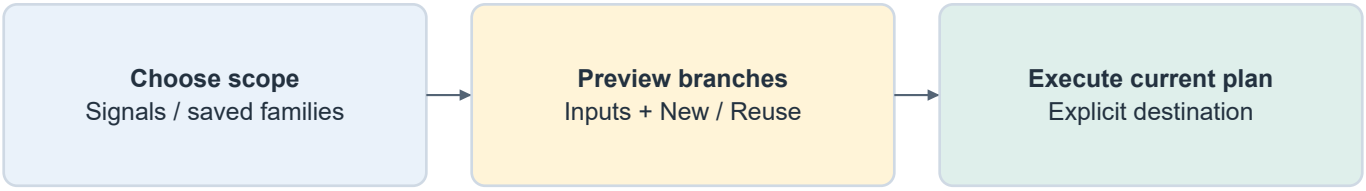


Figure 9. Selected raw/saved signals and ordered roles remain part of semantic calculation identity.

Automatic scopes use saved eligible objects: All Indicators, All Oscillators or All Constructs. Selected Signals can include permitted raw OHLC. Saved Volume eligibility follows the canonical tool policy. There is no implied Cartesian-product expansion.

Destination	Publication boundary
Individual Artifacts	Publish reviewed calculation results through the Artifact owner.
New / existing Artifact Collection	Validate intended outputs and exact destination head before Collection publication.
Cancellation / later failure	Earlier successfully published Artifacts or Recipes may remain; pre-commit cancellation preserves the old Collection head.

Successful execution invalidates the consumed preview. Batch uses the Suite foreground Operation surface and produces a separate terminal report; it is not a transaction across every pool.

Managed Artifact versions and updates

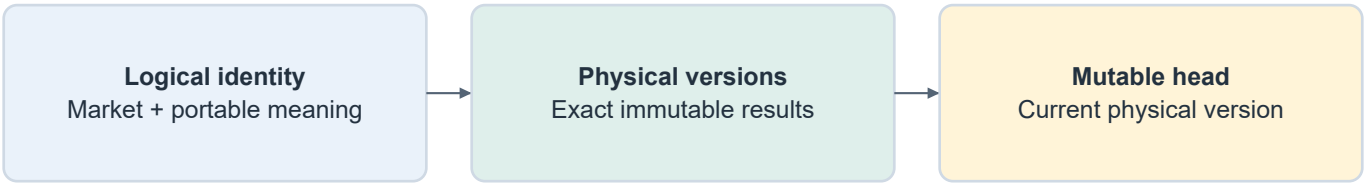


Figure 10. A new physical version preserves old results. A changed head does not rewrite consumers' copied values.

Operation	Rule
Inspect / load	Validate canonical metadata, values and lineage as required by the operation.
Assess currentness	Compare source/recipe/dependency evidence without recalculating or publishing.
Plan update	Use Financial Tools' canonical update policy, context requirements, overlap and revisable tail.
Execute update	Recheck plan evidence, calculate permitted scope, publish a new immutable version and advance the logical head.
Dependency update	Dependent calculations require explicit appropriate update; no hidden cascading rewrite.
Delete managed Artifact	Check genuine Artifact/Collection references. Deletion does not erase owned Database copies.

Artifact update is not Database append
Updating a provider can change its permitted current result. Existing Database cells, including historical NaNs, remain untouched. A subsequent Database append copies only its newly admitted suffix.

The saved Artifact embeds its market-bound execution recipe and exact lineage. Losing a separately persisted global Portable Recipe does not, by itself, make that payload invalid. Genuine dependencies remain subject to their own reference checks.

Metadata-only browsing and strict values loading are distinct operations. A catalog row is not proof that every value payload was just read and hashed.

Artifact Collection lifecycle



Figure 11. Collection membership records explicit physical versions and output mappings; it does not follow every future head.

1. Choose a nonempty root set. Resolve required supports, unique output column names and presentation order.

2. Check canonical market, row count and endpoints across materialized inputs, then validate payloads and dependency closure.

3. Publish a revision with member version keys, logical edges, selected outputs, source evidence and physical value provenance.

4. On membership edit, preserve retained exact versions. Pure removal does not refresh unrelated members; retain shared supports still required.

5. Advance only the Collection's head. Import into a Database is a separate explicit operation that copies values into that Database.

Later event	Consequence
Artifact head advances	Existing Collection revision stays fixed; assessment may report an update need.
Remove root	Remove only outputs/supports no longer justified by the final graph; preserve other members.
Delete Collection	Member Artifacts survive. Databases retain their owned copied values and historical attribution.
History inspection	Read exact earlier revisions without moving the head.

References and copied values differ
A live Artifact Collection still refers to its member Artifact versions. A Database has its own values, so the original Collection is historical import evidence rather than ongoing ownership of those cells.

Seed creation and append

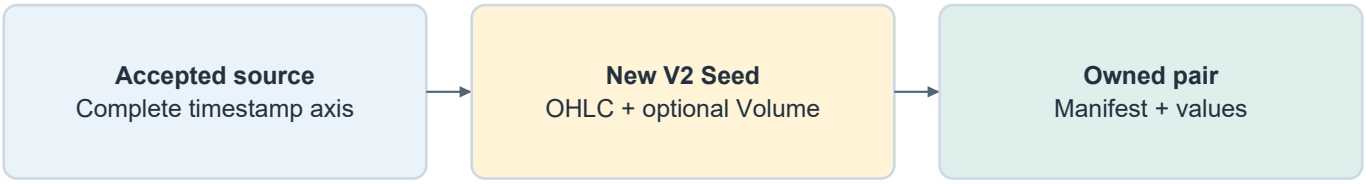


Figure 12. New creation saves complete source coverage. Legacy range fields do not expose arbitrary trimming in the current creation flow.

Rule	Behavior
Coverage	Full accepted row count and first/last timestamps; mandatory Open, High, Low and Close, optional Volume.
Historical retention	A saved Seed may remain older than the current source. It does not auto-follow new candles.
Explicit append	Verify unchanged old prefix and continuous new suffix; preserve every old row and cell.
No new rows	Return the existing Seed; publish nothing.
Legacy V1	Keep its reference-based format. Append does not silently migrate it to V2.
Ambiguous storage	If V1 file and V2 directory claim the same ID, fail closed rather than choose one.

V2 append publication and recovery

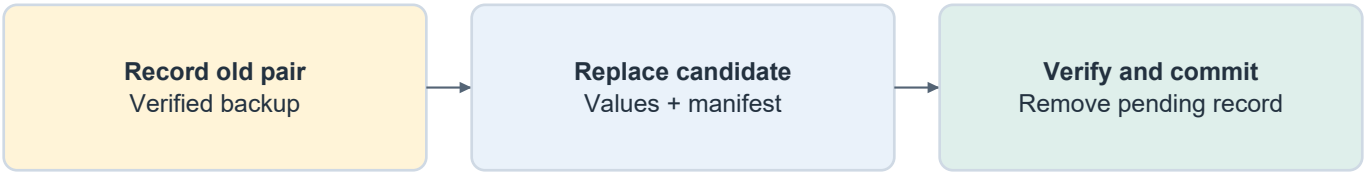


Figure 13. Pending reads expose the verified old pair. Failed publication restores it; post-commit backup cleanup can leave a disposable backup.

Deleting a Seed removes that Seed-owned persistence through its owner. Already created Databases retain their own definition, manifest and values; they are not views over a live Seed file.

Seed-only Database creation

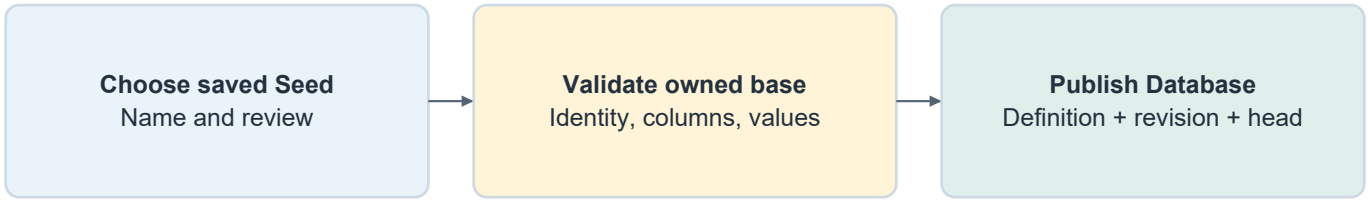


Figure 14. The current GUI first creates an owned base Database. Derived content is added explicitly afterward.

1. Open Catalogs > Databases > Create Database and select a saved Seed. The displayed source and coverage are part of the reviewed intent.

2. Validate the Seed through its format owner. V2 owns values; legacy V1 needs provable source coverage to obtain its base values.

3. Plan a new stable Database ID and immutable definition. Prepare a full base revision with ordered columns and exact values hash.

4. Recheck the reviewed Seed/evidence and cancellation boundary. Publish the immutable bundle before advancing the Database head.

5. Immediately project the successful result into Catalogs. Later background reconciliation verifies currentness without changing membership.

Owned after creation	Not implied
Database definition, revision manifest and values	Automatic following of the original Seed or accepted source
Immutable revision history and explicit head	A mandatory Environment / Recipe / Collection pipeline
Base OHLC and optional Volume	Automatic derived columns or implicit recalculation

Full source axis is intentional
Keep legitimate missing values only where the format permits them. Base values are finite; derived warm-up NaNs later imported into the Database remain on the full axis.

Database membership and ownership

An explicit edit starts from the Database's own values and final intended ownership. It does not reconstruct retained columns by reloading every historical provider.

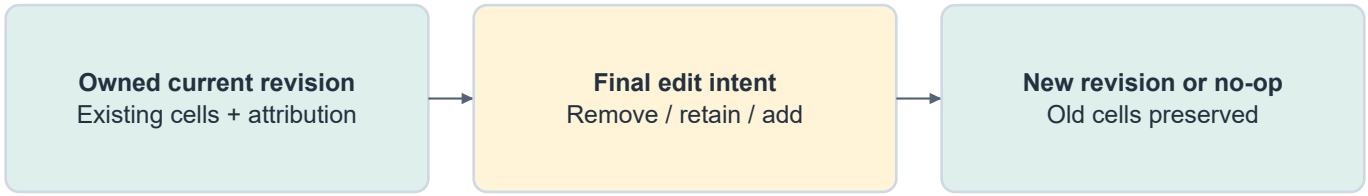


Figure 15. Incoming-value validation is separate from retention of already owned values.

Action	Meaning
Add Artifact or Collection	Validate only the required incoming compatible payload and dependencies, then copy reviewed columns.
Remove a source assignment	Drop that owner. A column survives if another retained owner still justifies it.
Remove a derived output	Remove that saved column from every ownership assignment that selected it.
Base columns	Remain protected from derived-membership removal.
No-op	Return existing state without a new revision or writes.
Deleted upstream provider	Does not block local no-op, removal or unrelated compatible additions merely because old values once came from it.

Example: two owners, one column

A direct SMA Artifact supplies sma_3. Later a Collection supplies the same SMA plus ema_4. Removing the Collection assignment removes ema_4, while sma_3 remains because its direct Artifact assignment survives. Removing sma_3 itself removes that column across all assignments.

Editing has a different proof obligation from growth
Local retention needs truthful owned values and attribution. Future append needs a provable current provider for each retained derived output. These operations must not share an overbroad upstream-existence prerequisite.

Append and truthful legacy history

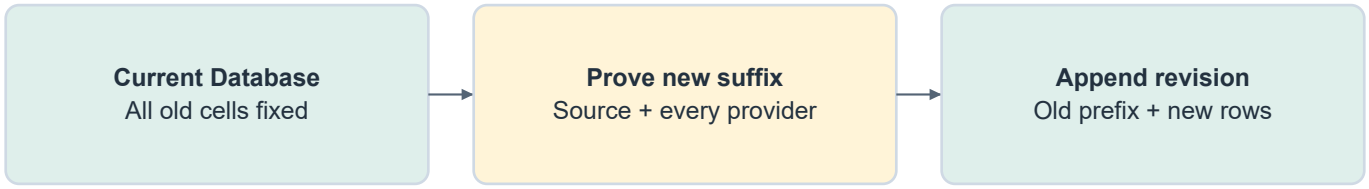


Figure 16. APPEND adds a continuous suffix. CURRENT writes nothing. BLOCKED reports the missing proof.

1. First decide whether accepted source growth exists. No new rows means no new publication.

2. For growth, prove every remaining derived column's future provider and coverage. Explicit Artifact updates may be needed first.

3. Preserve all historical values and NaNs. Reject historical source mutation, shrinking, broken continuity or an unprovable provider as required by the plan.

4. Combine old and new physical provenance segments, recheck source/heads/bindings, then publish immutable values and manifest.

Revision version	Meaning
V1 / V2	Legacy imports may lack exact column-to-object or ownership proof.
V3	Explicit source ownership of selected columns.
V4	Normal fully attributed publication; physical provider segments preserve append history.
V5	Approved Database-only extension for explicit operations retaining genuinely unknown legacy attribution. Captures original manifests; invents no IDs.

Unknown stays unknown
A surviving historical candidate is not enough to relabel old cells. Exact future segments can prove a later provider after original legacy coverage, without pretending that provider is proven for the past.

V1-V4 old files remain unchanged. Rebuild is retired and the rebuild endpoint refuses. Resolve an append blocker explicitly: remove the affected column or establish a provable provider and prepare a new plan.

Removal and deletion survival

Operation	Surviving state / protection
Delete Environment	Derived Recipes and saved calculations survive; Research retains Environment authority.
Remove Recipe from Collection	Global Recipe survives; historical Collection revisions may still protect it.
Delete Recipe Collection	Member Recipes survive; Collection-owned history is removed through its owner.
Delete Portable Recipe	Genuine Recipe references can block it. Embedded valid Artifact execution evidence remains independent.
Remove Artifact from Collection	Pool Artifact survives. Required shared supports remain in the final valid graph.
Delete managed Artifact	Genuine Artifact/Collection references are checked. Owned Database values survive; future append may become blocked.
Delete Artifact Collection	Member Artifacts and Database-owned copies survive.
Delete Seed	Already created Databases retain their owned base values and history.
Remove Database assignment	Only that ownership relationship is removed; multiply owned columns can remain.
Delete Database	Delete only verified Database-owned definition/head/revisions; preserve upstream Seeds, Recipes, Artifacts and Collections.

Deletion is not garbage collection
No pool operation silently performs a general cascade across every object that once contributed data. Inspect the real owner and surviving references before interpreting a missing upstream object as corruption.

Duplicate Maintenance previews canonical semantic equivalence and protected references. It does not merge by display name, guess missing legacy identity, rewrite historical values or create a new universal object manager.

Reconciliation and currentness

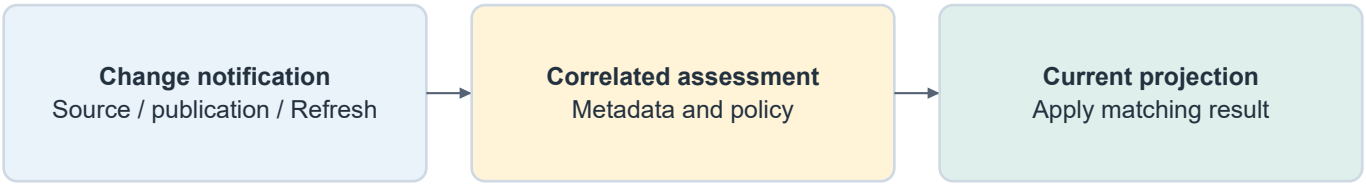


Figure 17. Reconciliation derives state. It never silently calculates or publishes a new product.

Situation	Required response
First Suite initialization	Foreground warm-up coordinates reconciliation and catalog projection; failure does not falsely mark success.
After warm-up	Use separately correlated background reconciliation; coalesce requests and apply only current results.
Successful mutation	Immediately project committed result, invalidate stale assessment and schedule verification.
Source or selection changes	Invalidate affected evidence/generation and reject stale results for the previous context.
Window disposed / run superseded	Ignore presentation delivery without inventing a different Core terminal result.

Vocabulary carries different claims

Valid: canonical payload and required evidence satisfy their format. **Current:** assessed against present source and dependencies. **Compatible:** admitted under the three alignment rules and applicable input contracts. **Owned:** values belong to the saved object. None of these words substitutes for the others.

Assessment is not permission for a retired operation
Historical enum values may still represent REBUILD_REQUIRED. Current Database planning uses CURRENT, APPEND or BLOCKED. It does not expose an automatic rebuild, backfill or shrink path.

Cancellation and window lifecycle

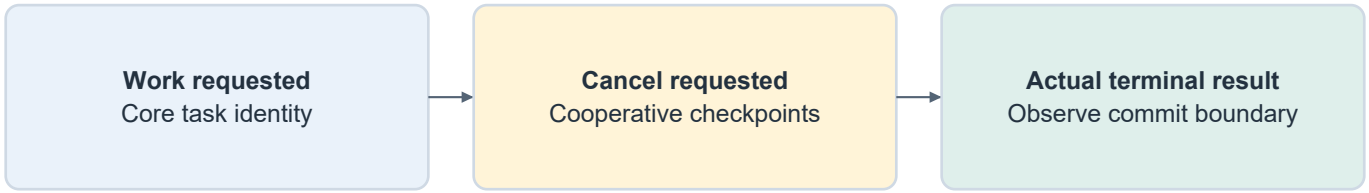


Figure 18. A cancellation request is not a rollback guarantee. Already committed objects remain truthful durable results.

Boundary	Behavior
Foreground work	One pending foreground owner; Operation progress belongs to that matching task.
Close Suite	Stop reconciliation scheduling, close bounded children, end check session, request cancellation and dispose presenter.
Late callback	Check disposal, generation, selection, task and retained dialog/plan; release only matching pending state.
Reopen retained Suite	Focus and preserve geometry. A disposed/new Suite receives new presentation/session context.
Window identities	Database Artifact editor and Collection editor use different IDs. Similar titles do not make them the same runtime window.
Open Notebooks / Research close	Research owns Notebook dirty-change and restoration admission. DMS navigation does not bypass it.
Application shutdown	Core owns shared termination and tracking. GUI accepted-closure state must reflect whether close was actually accepted.

Publication granularity matters
A cancelled Batch can leave earlier completed Artifacts. A Seed append has its own recoverable pair protocol. A Database revision has its own publication boundary. Do not claim a universal all-pool rollback.

Restart loads saved heads/history through canonical stores. It does not resume an obsolete GUI plan, infer unsaved work from temporary presentation state, or silently migrate old formats.

Integration with Leonardo

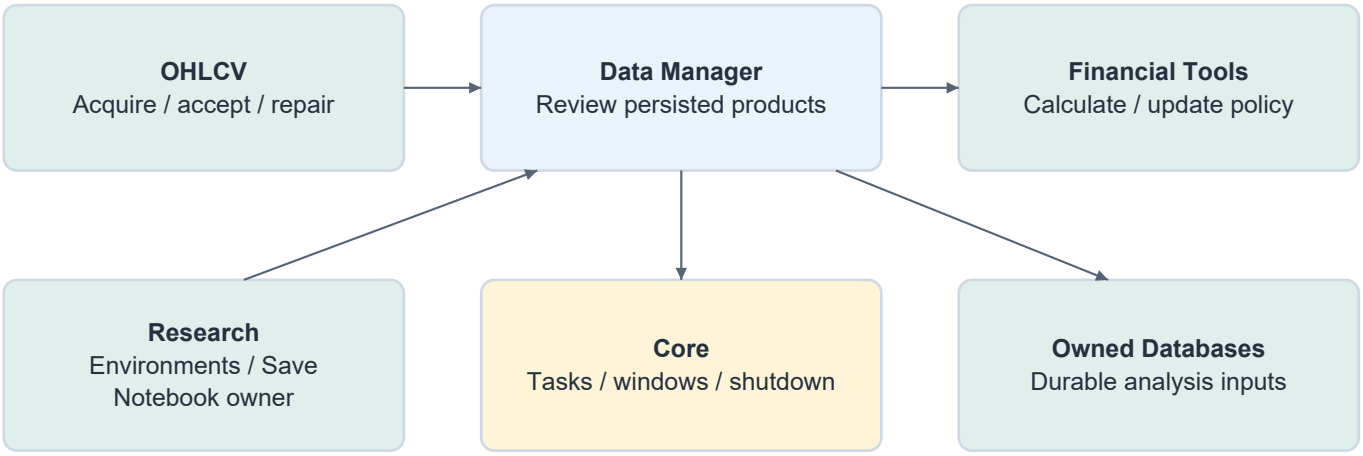


Figure 19. Arrows show selected dependencies/data paths. Adjacent Areas retain their canonical authority.

Integration	Contract
Research -> DMS	Environment definitions can derive Recipes; Research Save publishes through canonical Artifact/Recipe services. DMS reads current pool truth.
DMS -> Research	Dataset handoff carries complete MarketId. Open Notebooks focuses the actual Research Notebook Manager through shared composition.
OHLCV -> DMS	Accepted metadata and checked data feed explicit operations; source changes invalidate relevant DMS assessment.
Financial Tools -> DMS	Parameters, input eligibility, outputs, formulas, context and update strategy remain authoritative.
Core -> DMS	Execution, progress, cancellation, runtime/window identity and shutdown remain shared infrastructure.
Database -> downstream work	Saved values are durable inputs. This map does not claim a completed Analysis engine.

Workspace Snapshots, Notebook assignments, annotations and dirty-change handling stay in Research. A DMS shortcut does not create another Notebook lifecycle or persistence format.

Persistence and reconstruction

Canonical area	Durable structures
Historical market root	ohlcv candles and sidecar; market-bound recipes; Artifacts values/meta/optional analysis; logical version records and heads.
Recipe root	Portable definitions; separate provenance/metadata; Recipe Collection head and immutable revisions.
Data Manager root	Artifact Collection revisions/heads; V1 Seed file or V2 manifest/values pair; Database definition/head and revision bundles.
Research roots	Environment, Notebook and Snapshot formats retain Research ownership.

Bytes are part of the contract

Canonical JSON validates exact fields, identity and representation. Recipe/DMS revision hash input uses its indented sorted JSON convention; Artifact identity hashes use compact JSON. Do not hash whichever JSON formatting happens to be convenient.

Artifact CSV begins with ts_ms and declared output order. Numerical tool outputs use the canonical float32 round-trip format; permitted NaNs are empty tokens, booleans are lowercase and categorical outputs keep declared strings. Base Database values remain finite.

Reconstruct from the whole set
D contains 31 persisted-model inventories covering 329 declared/inherited attributes, exact model-to-wire differences and five generated complete payload examples. A defines dependencies. L defines state transitions and failure rules. S supplies the active interface and source/test map.

1. Recreate canonical identity/format validation and owned stores, including legacy read compatibility and immutable history.

2. Integrate existing OHLCV, Research, Financial Tools and Core boundaries; preserve their authority.

3. Implement explicit plans, checks, publication, membership and update policies before the GUI projections.

4. Validate observable lifecycle outcomes, then add presenter correlation, windows and navigation. Use canonical tool specifications for formula-level implementation.

Evidence and present limitations

Evidence	Precise scope
Completed DMS final audit	2,720 full-suite passes / 7 skips; six matching isolated native checks reused after candidate comparison.
Later Open Notebooks shortcut	43 focused tests and 8 isolated Windows checks. The earlier 2,720-test result predates this change.
Later Research A4 audit	2,831 full-suite passes / 7 skips on the checkout including that shortcut; later 115 offscreen and 21 native checks. Counts overlap.
Original 8 October documentation	295 existing tests passed; generated model/value examples, current source review, link/field checks, rendered map inspection and exact PRE/POST preservation. No new full-suite or native application claim.

Long paths: R02 code/package resolution, 27 October
DMS-DOC-F1 (alias XA-DMS-F003) was repaired through canonical read/enumeration/delete I/O and demonstrated ordinary-long-root rollback. The 8 October reproduction remains historical. Six file-symlink cases, actual UNC and native/registry behavior remain unexecuted; arbitrary-depth writer support is not claimed.

The earlier intermittent Windows CSV replacement cause remains unidentified. Controlled containment and existing Maintenance recovery were verified; Update Latest can leave an interior gap that needs reviewed Maintenance repair. The long-path finding does not establish that earlier cause.

R07 changes documentation only. R08 integrated and R09 native acceptance remain pending. No universal filesystem atomicity or new storage/large-file behavior is claimed.

Canonical evidence and maintenance rule

Original evidence: Leo_V2_Task_Packages/20261008_DMS_Documentation_Complete_Map. R02 resolution: 20261027_T1097_R3_REVIEW. R07 document checks: 20261028_T1102_R07_R1. Counts above retain their original candidate/edition attribution.

When approved behavior changes, update the relevant detailed contract/lifecycle first, then the map and evidence links. A visual simplification must never create a new authority, conceal unknown history or override a canonical validator.